

Building communication platforms for the IoT

Troels Brødsgaard, Uptime

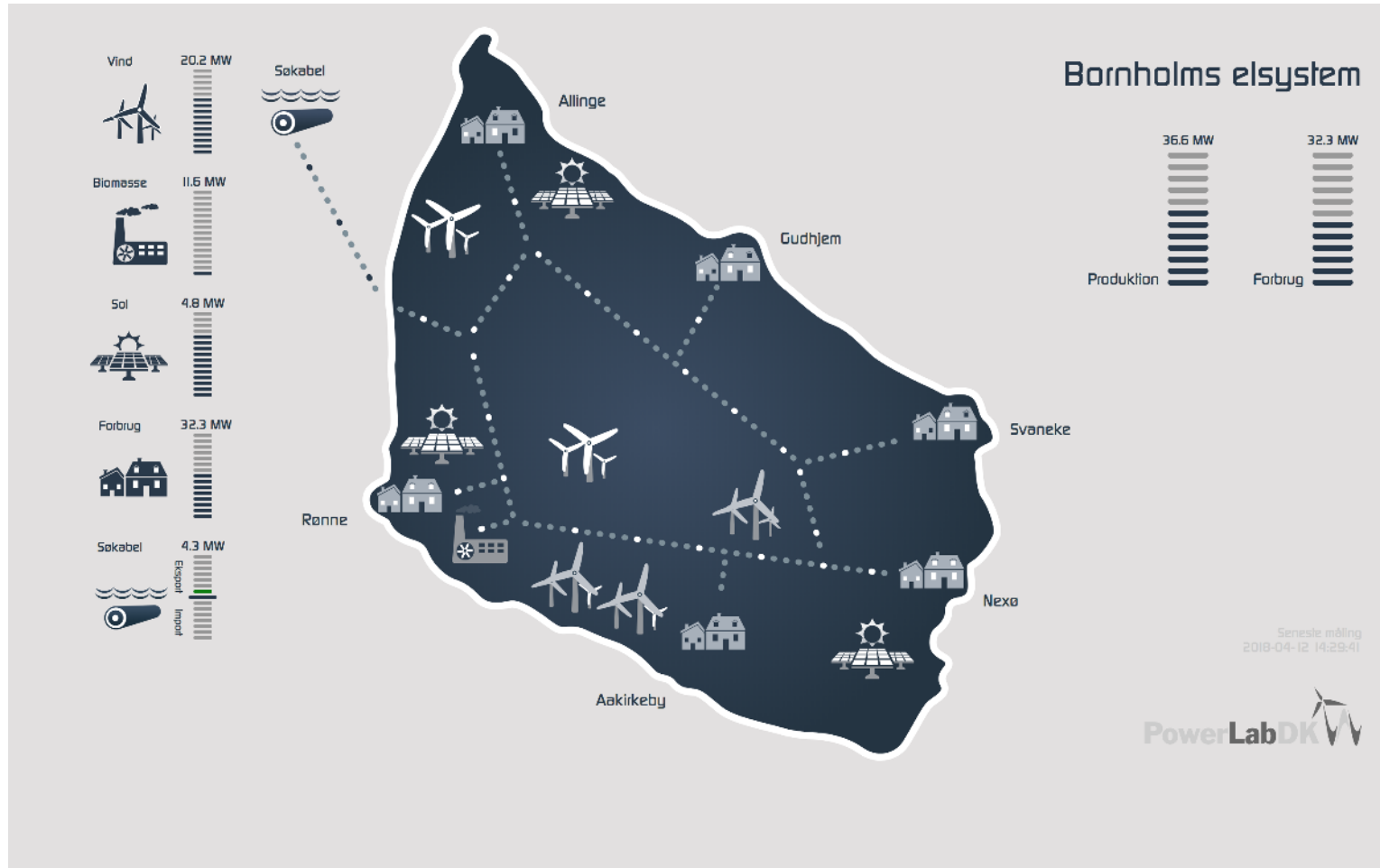
An island not so far away



An island not so far away



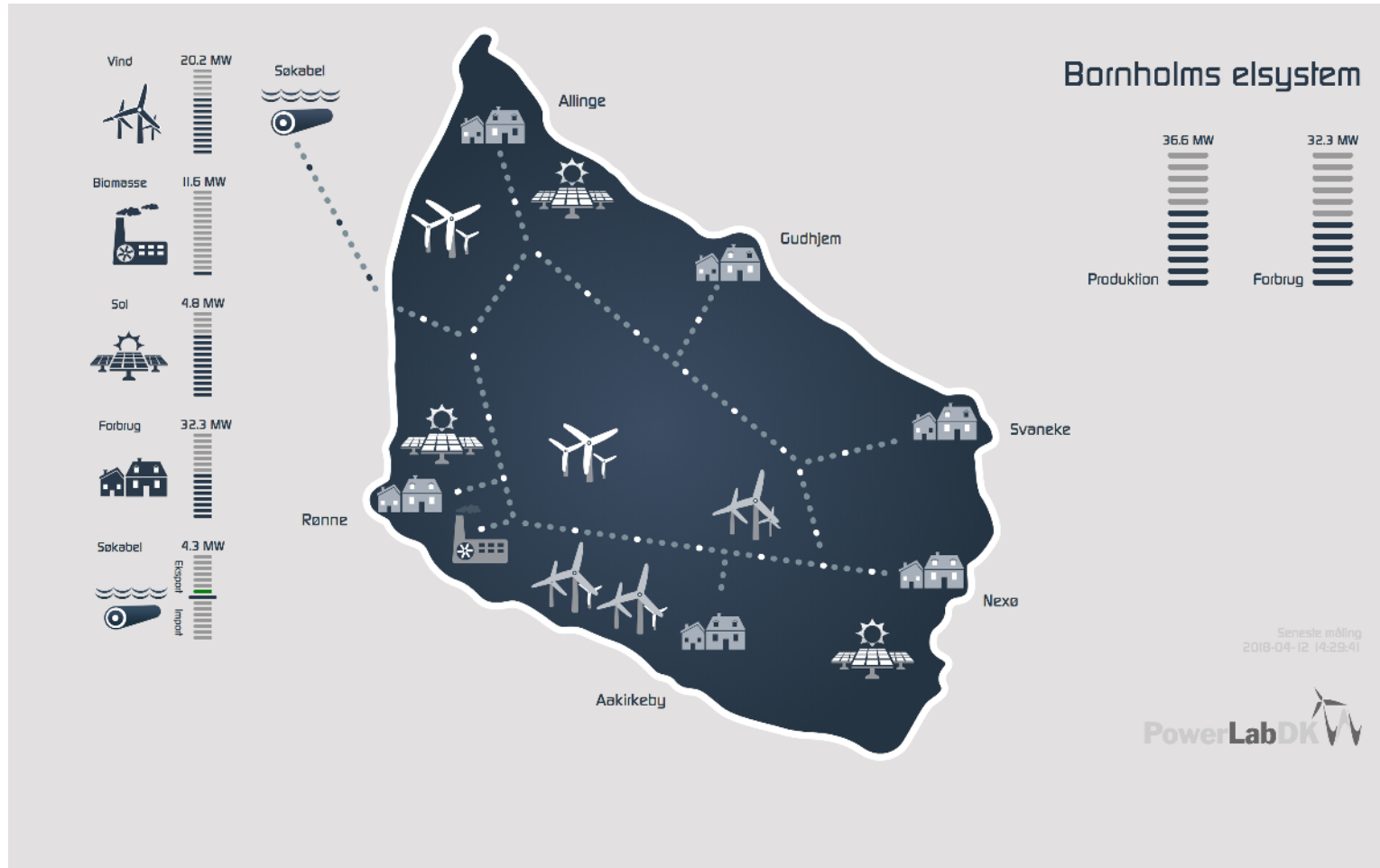
Bornholm Power System



Good old days



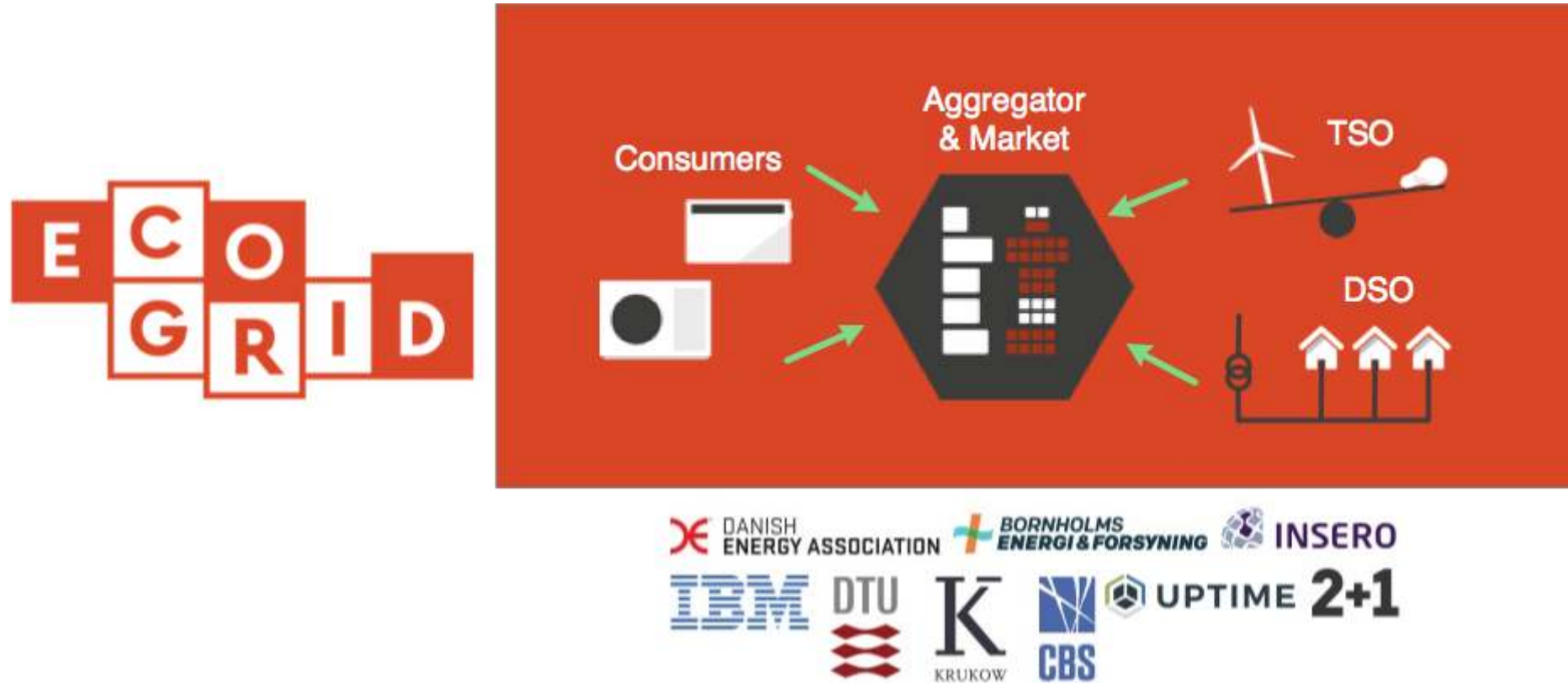
Brave new world



Flexible consumption - DER



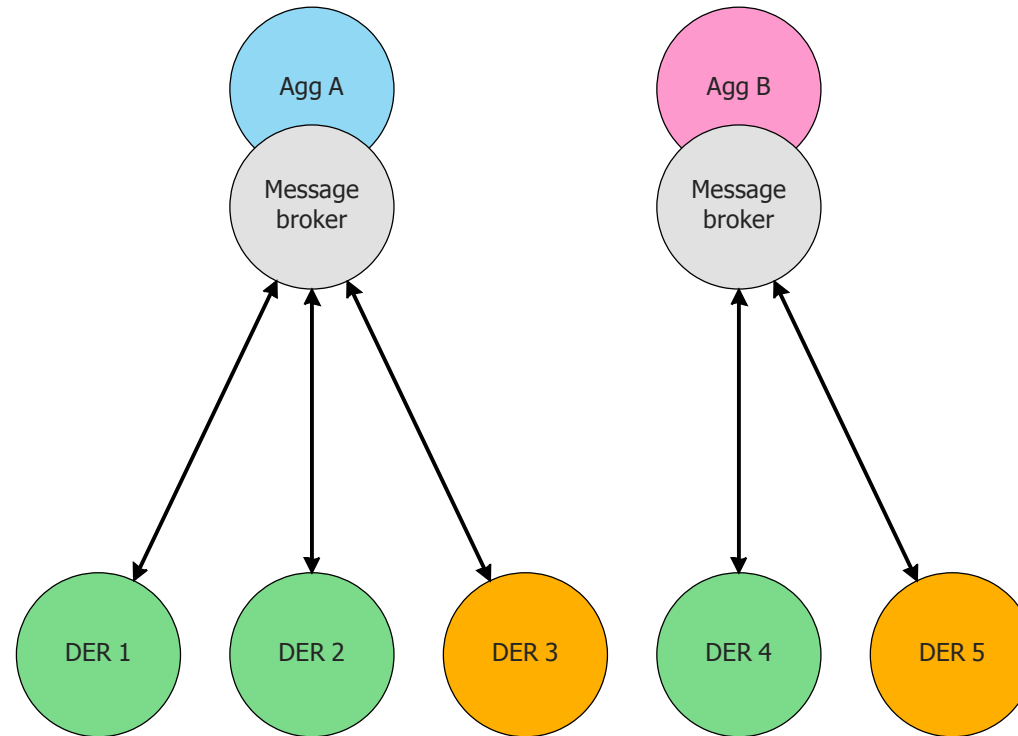
The Ecogrid 2.0 approach



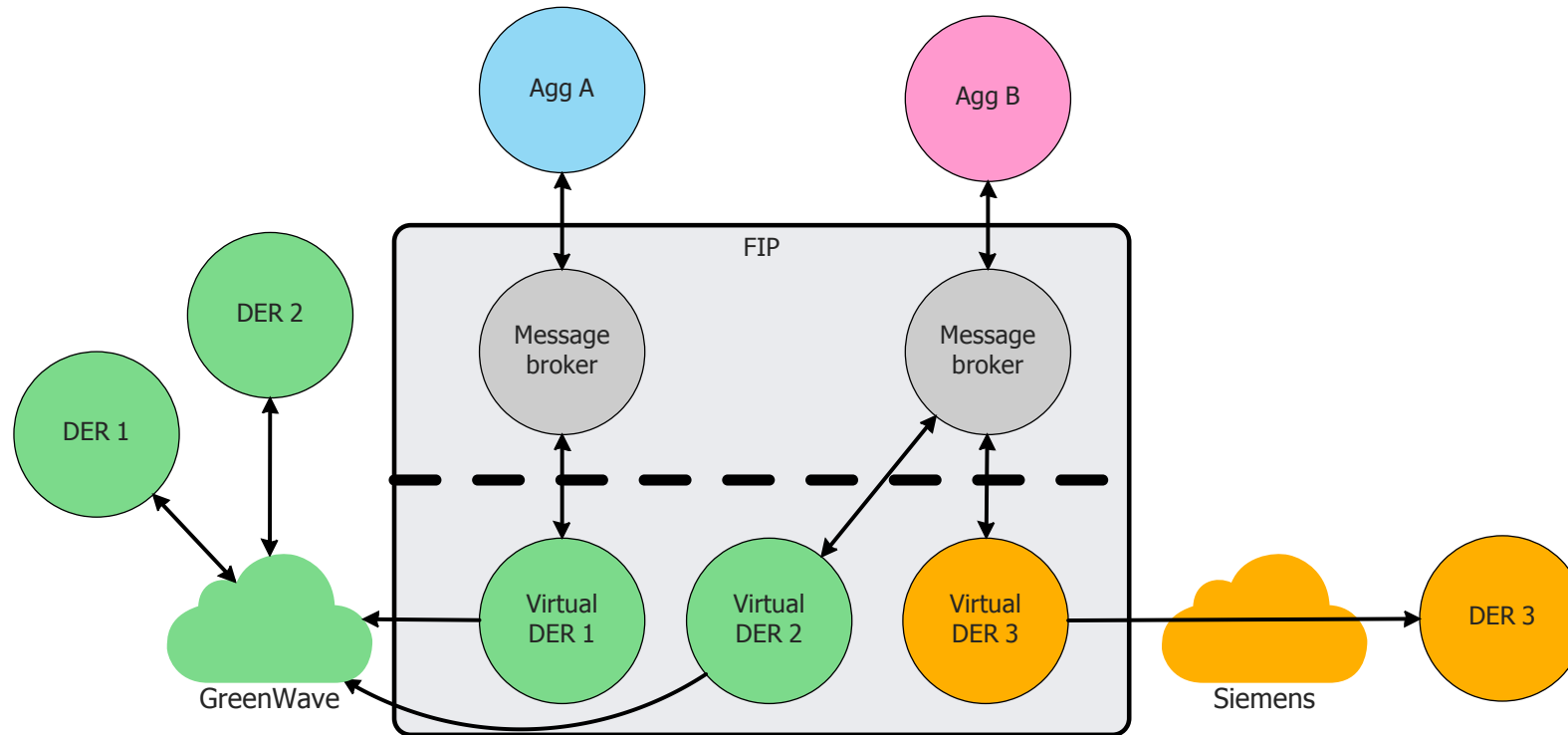
Interoperability



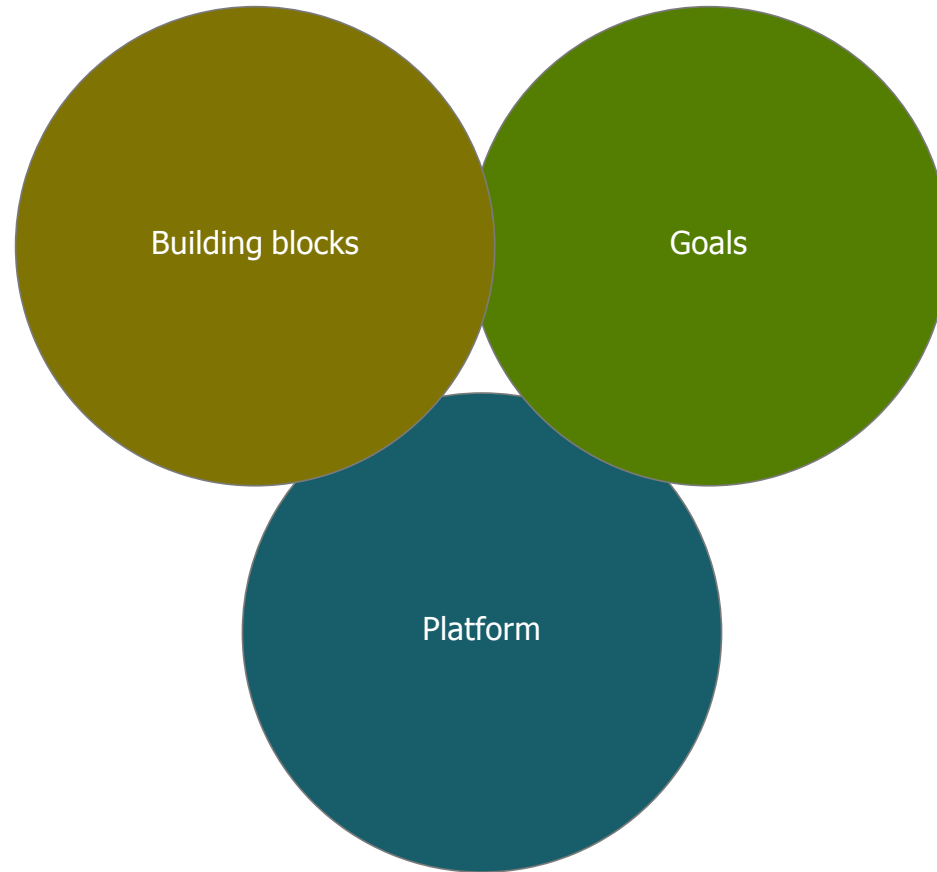
The ideal



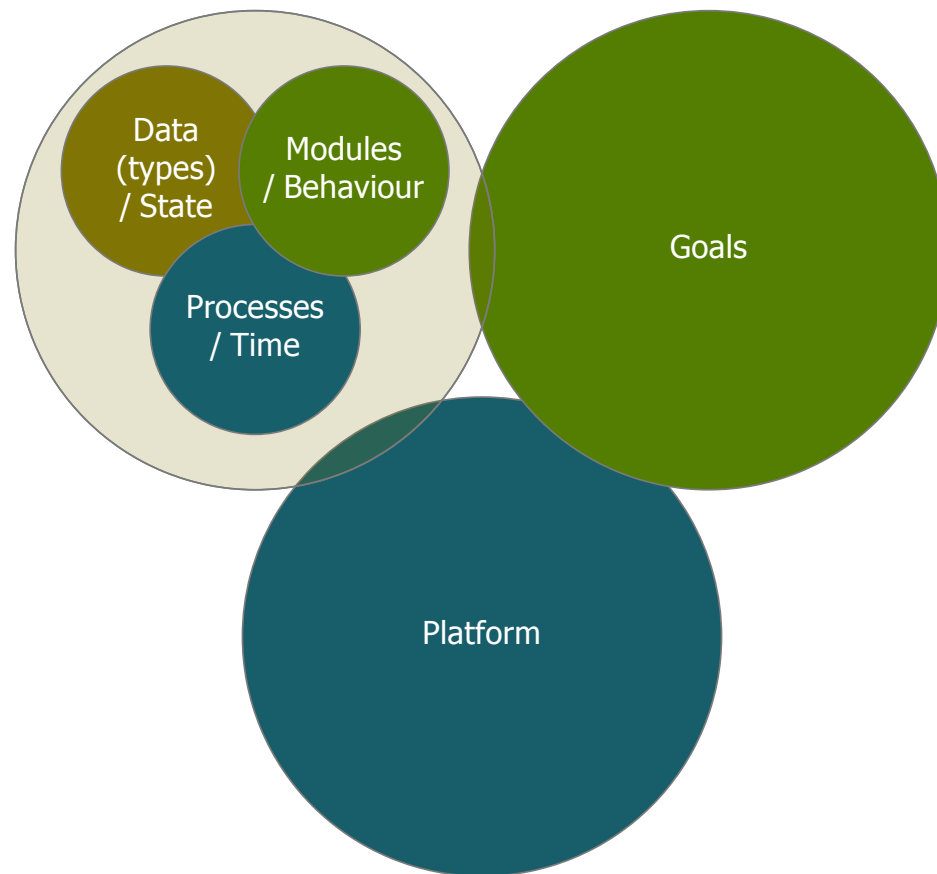
Flexibility Interoperability Platform



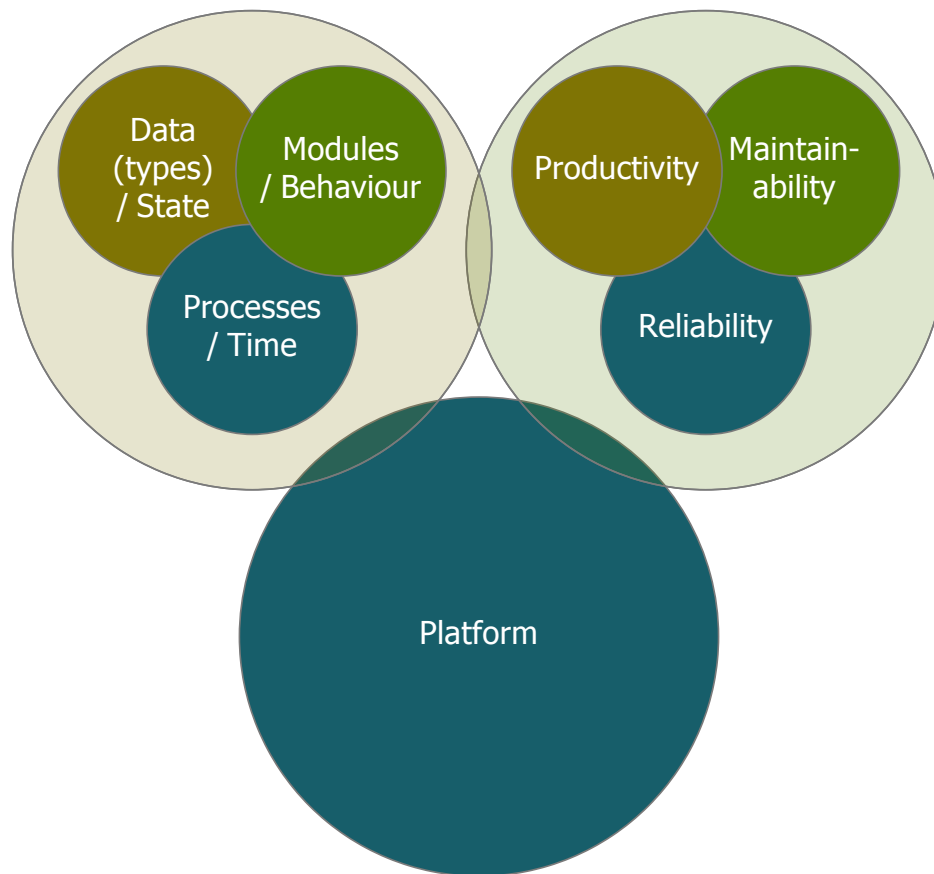
Why Elixir?



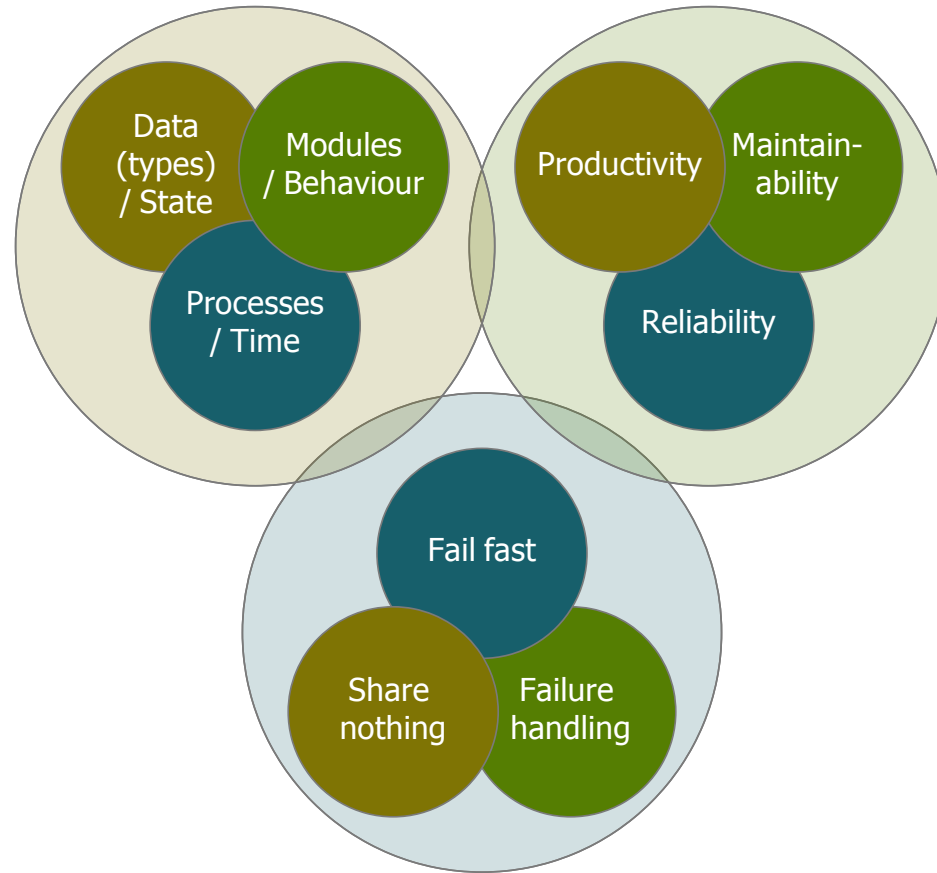
Why Elixir?



Why Elixir?



Why Elixir?

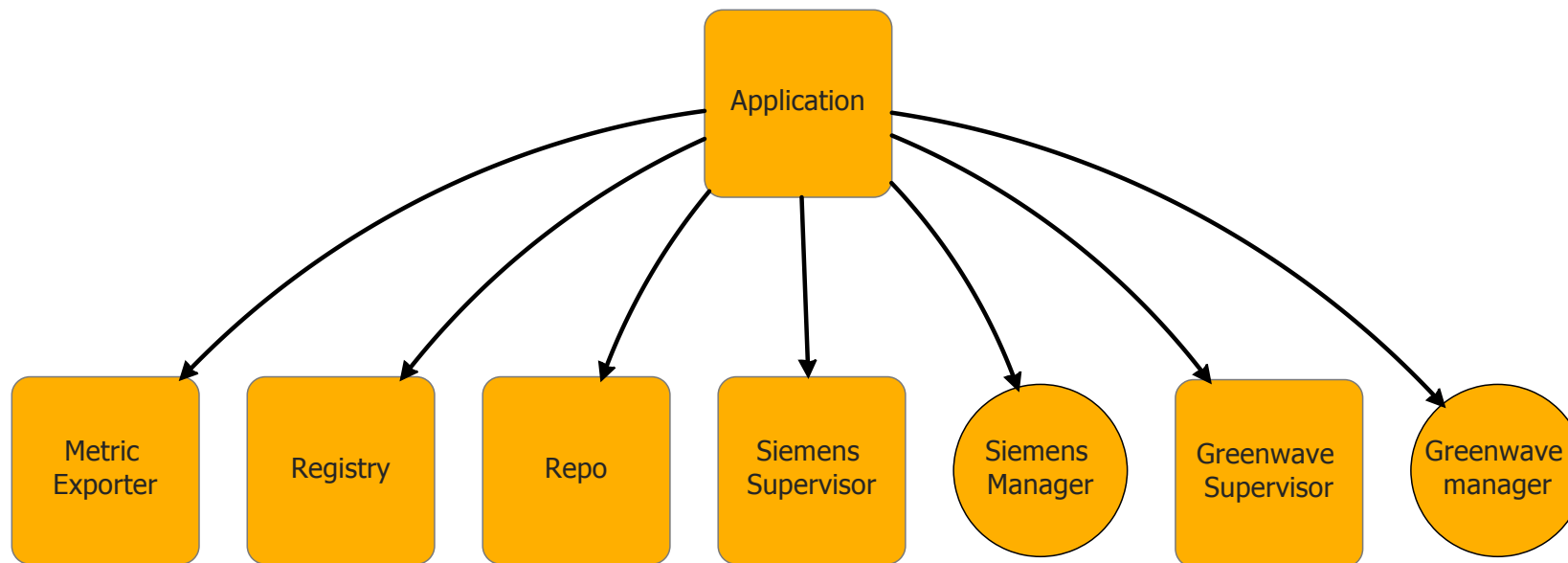


What I want to share

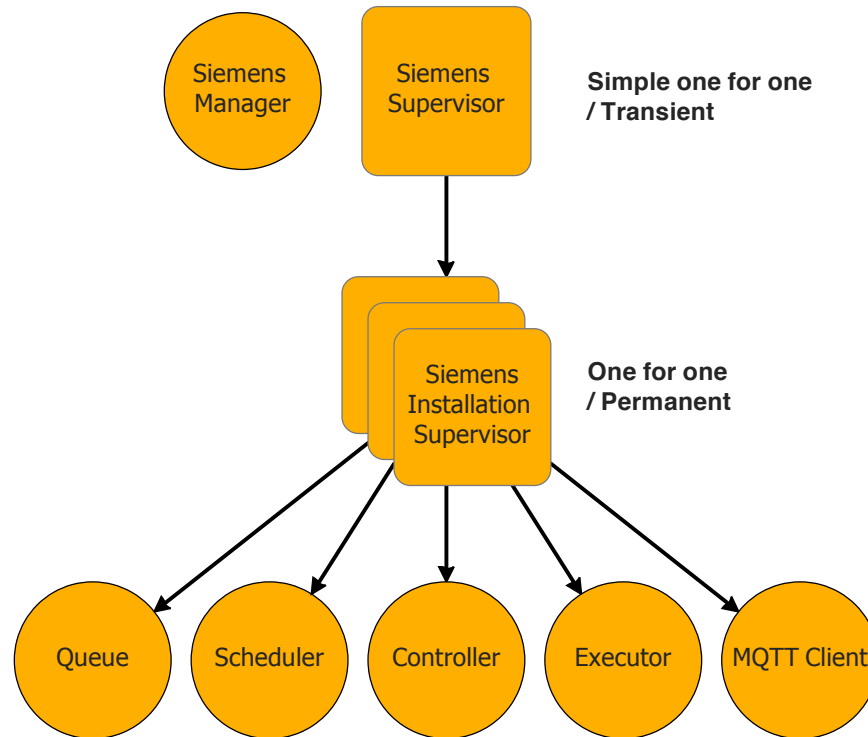
- Supervising dynamic processes
- Isolated but collaborating processes
- Expecting the unexpected
- Supervising based on error kernels
- Returning errors
- Instrumentation and monitoring
- Deployment
- Debugging
- In summary
 - Things we did right
 - Things we would have done differently

Supervising dynamic processes

Application overview



Siemens VDER subtree



Where did my processes go?



This looks suspicious

```
=SUPERVISOR REPORT==== 7-Mar-2018::15:13:08 ===  
  Supervisor: {<0.2243.0>,'Elixir.FIP.Siemens.InstallationSupervisor'}  
  Context:    shutdown  
  Reason:     reached_max_restart_intensity
```

What do the docs say?

Supervisor

Notice that supervisor that reached maximum restart intensity will exit with `:shutdown` reason. In this case the supervisor will only be restarted if its child specification was defined with the `:restart` option set to `:permanent` (the default).

What do the docs say?

Supervisor

Notice that supervisor that reached maximum restart intensity will exit with `:shutdown` reason. In this case the supervisor will only be restarted if its child specification was defined with the `:restart` option set to `:permanent` (the default).

Supervisor.Spec

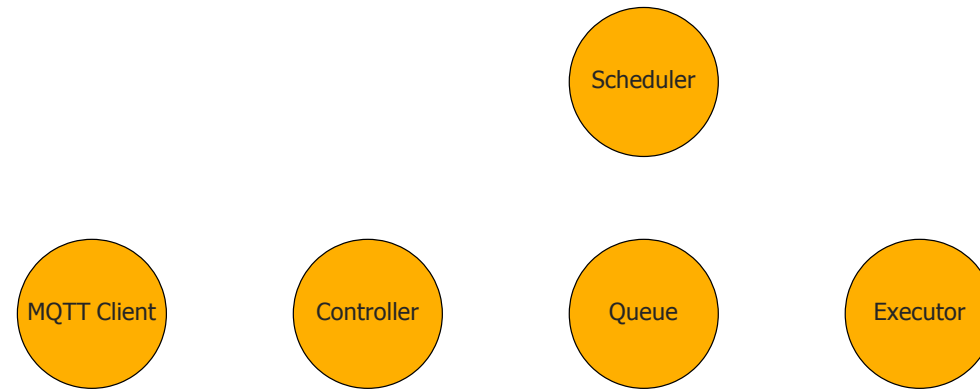
`:transient` - the child process is restarted only if it terminates abnormally, i.e., with an exit reason other than `:normal`, `:shutdown` or `{:shutdown, term}`

Takeaway - Supervising dynamic processes

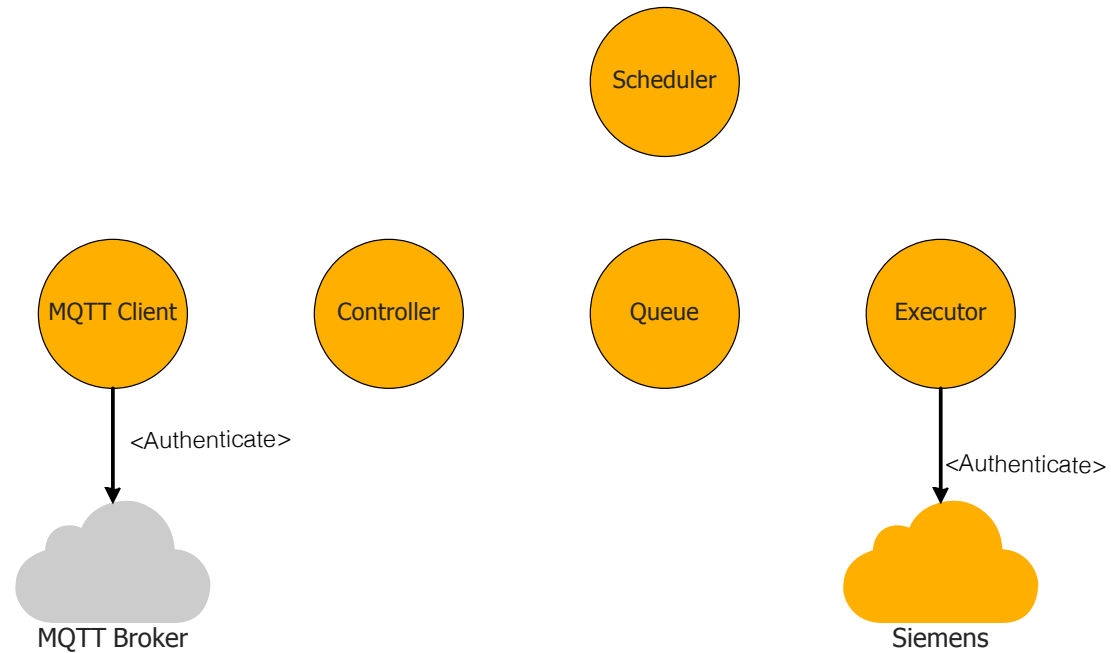
- Dynamic supervisors can't handle all failures
- The manager has to keep an eye on the running processes
 - It should keep an eye on the InstallationSupervisors, check regularly if any are missing
 - If it finds one that is missing, start it again
 - If it keeps dying, try with exponential backoff with jitter - and report the error somewhere.
 - See Configuration Manager in [The Hitchhiker's Guide to the Unexpected](#) by Fred Herbert

Isolated but collaborating processes

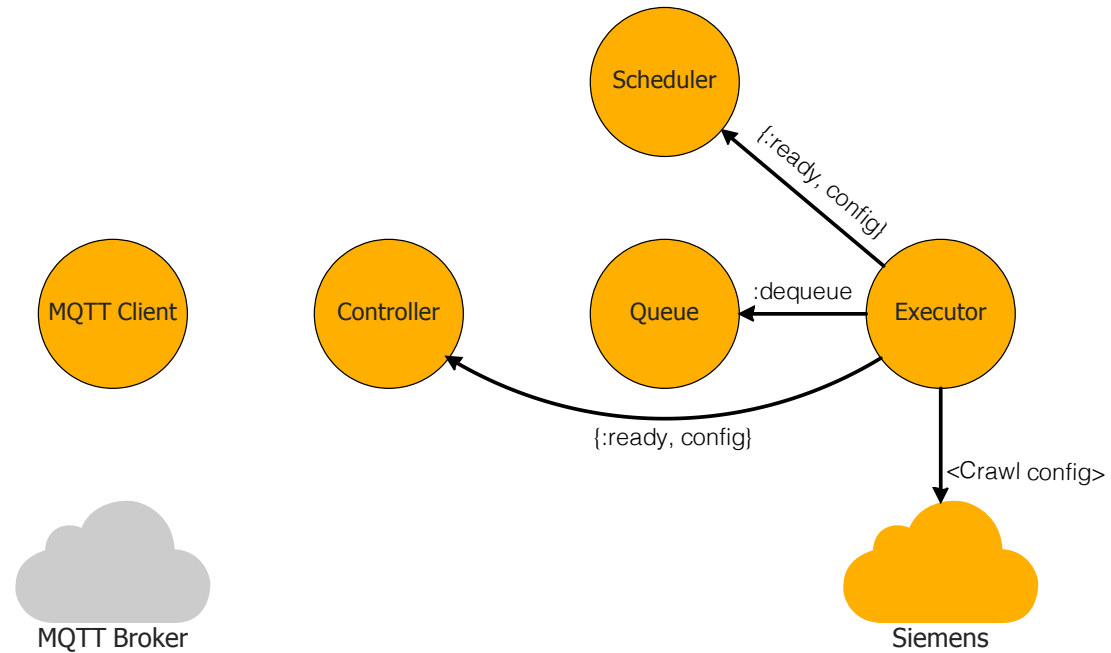
Siemens installation



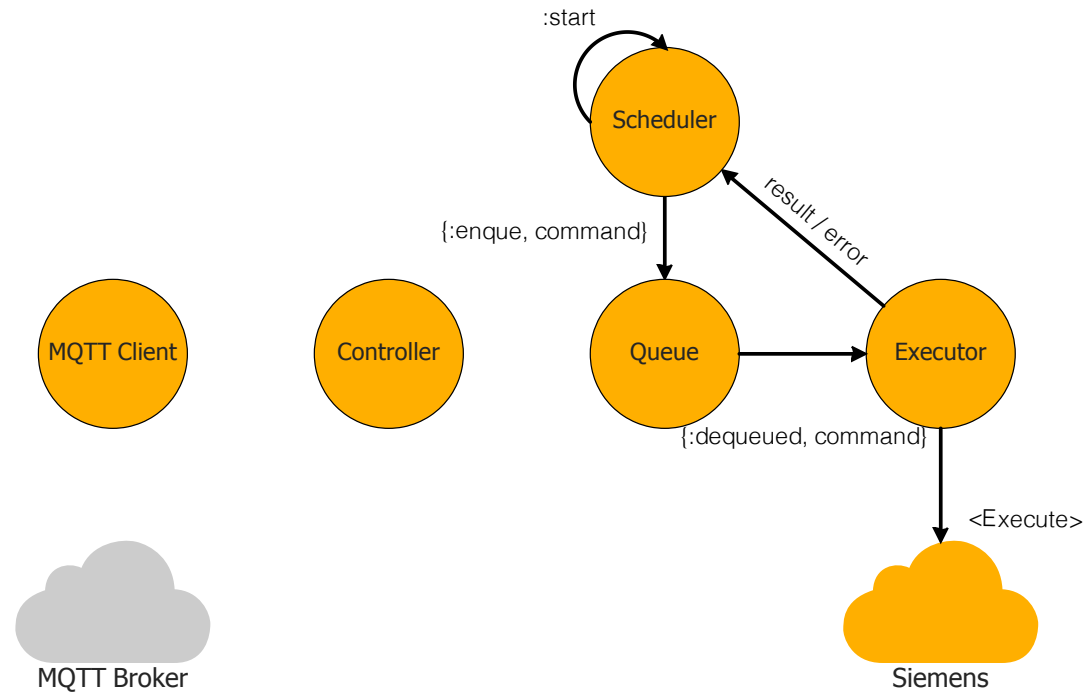
Siemens boot process



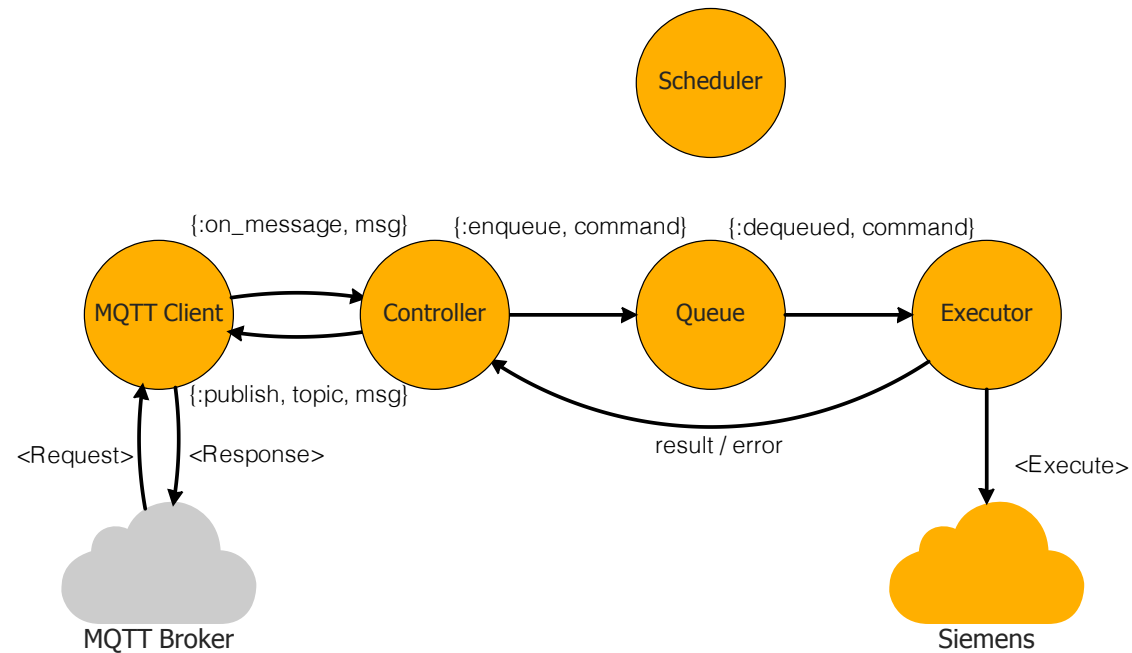
Configuration crawling



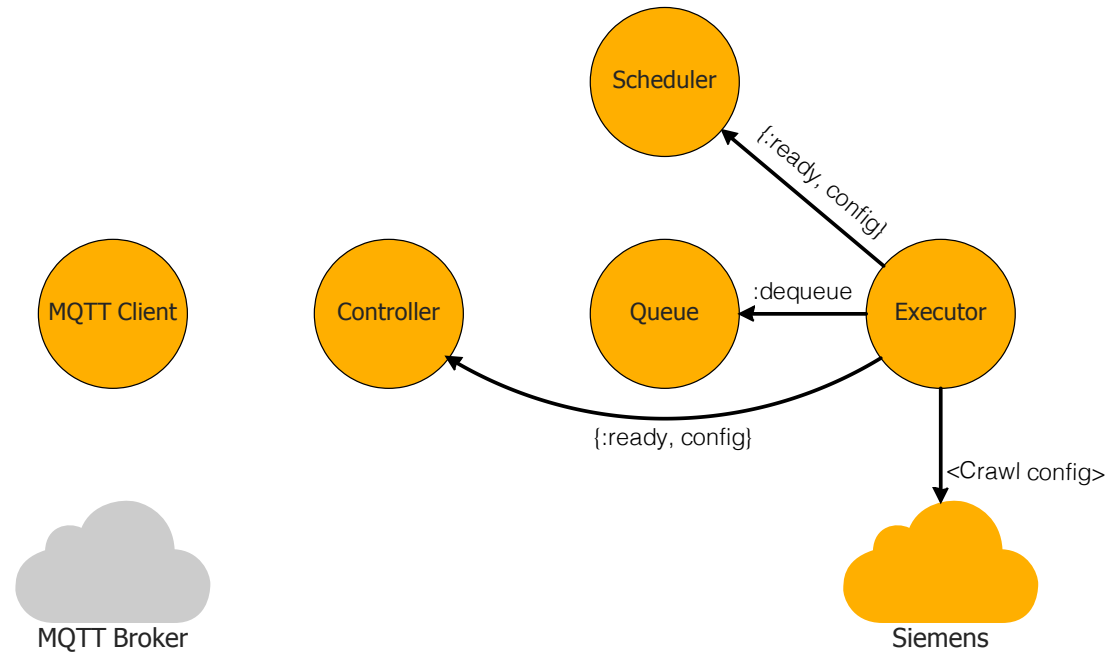
Scheduled background execution



Handling MQTT messages



Oops: what if the Scheduler is restarted?



Takeaway - Isolated does not mean independent

- If you're putting collaborating processes below the same `:one_for_one` supervisor, make sure they don't depend on each others initialization step
- If they really must depend on each other, you're probably looking for a `rest_for_one` supervisor.

Process registration

```
defmodule FIP.Siemens.Controller do
  alias FIP.Registry

  def init(installation) do
    der_id = "siemens_#{installation.id}"
    {:ok, _} = Registry.register(der_id, :controller)
    ...
  end
end
```

```
defmodule FIP.Registry do

  def register(der_id, role), do: Registry.register(__MODULE__, der_id, role)

end
```

Process lookup

```
defmodule FIP.Siemens.Controller do
  alias FIP.Registry

  def handle_call({:woopsa, %{"Header" => %{"Verb" => "write"}} = msg}, _from, state) do
    ...
    {queue, _} = Registry.find(state.der_id, :queue)
    ...
  end
end
```

```
defmodule FIP.Registry do

  def find(der_id, role) do
    __MODULE__
    |> Registry.match(der_id, role)
    |> List.first()
  end
end
```

Dispatch

```
defmodule FIP.Siemens.Executor
  defp announce_ready(state) do
    ...
    Registry.dispatch(state.der_id, fn (entries) ->
      for {pid, name} <- entries,
        Enum.member?(@announce_ready_to, name) do
        send(pid, {:ready, state.config})
      end
    end)
  end
end
```

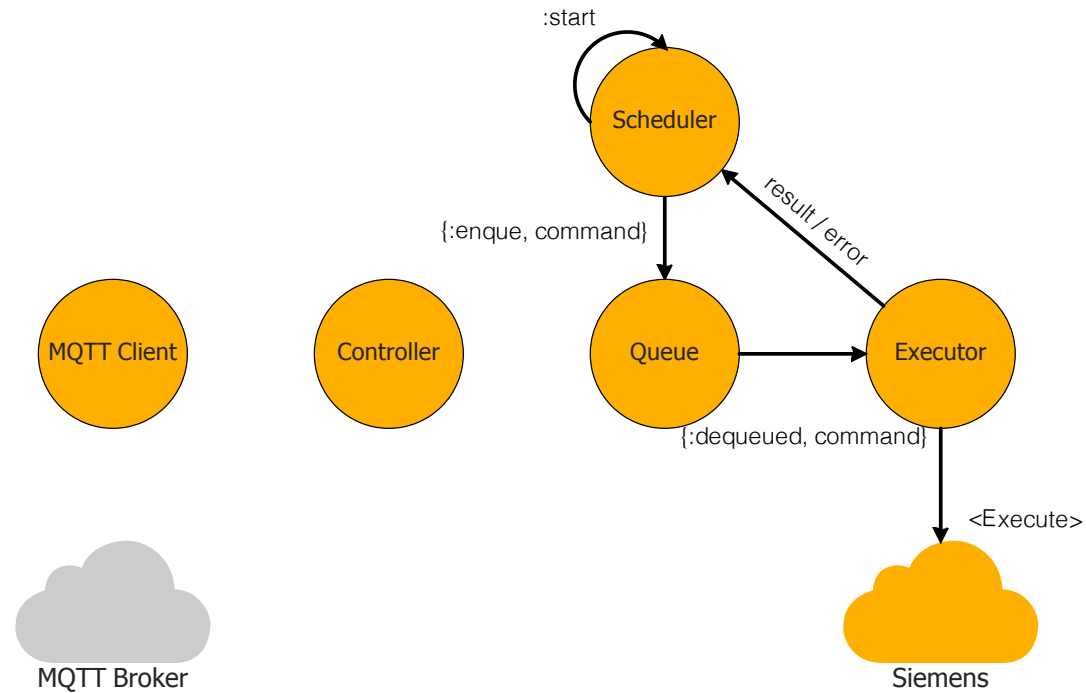
```
defmodule FIP.Registry do
  def dispatch(der_id, fun), do: Registry.dispatch(__MODULE__, der_id, fun)
end
```

Takeaway - Use the Registry

- It works really well for single-node apps
- It's really easy, has a nice API
- Using it as a PubSub allows you to add processes with additional functionality
- For multi-node, definitely look at Phoenix.Tracker and Phoenix.PubSub

Expecting the unexpected

What is that Scheduler doing?



Beware functions which look safe

```
defmodule FIP.Siemens.Scheduler
  def finish_job(job_state) do
    ...
    # Repo.insert/1 should return {:ok, schema} or {:error, reason}
    # while Repo.insert!/1 throws an error
    FIP.Repo.insert(new_sample)
    ...
  end
end
```

Beware functions which look safe

```
defmodule FIP.Siemens.Scheduler
  def finish_job(job_state) do
    ...
    # Repo.insert/1 should return {:ok, schema} or {:error, reason}
    # while Repo.insert!/1 throws an error
    FIP.Repo.insert(new_sample)
    ...
  end
end
```

initial call: Elixir.FIP.Siemens.Scheduler:init/1

pid: <0.17406.183>

registered_name: []

exception exit: {timeout,

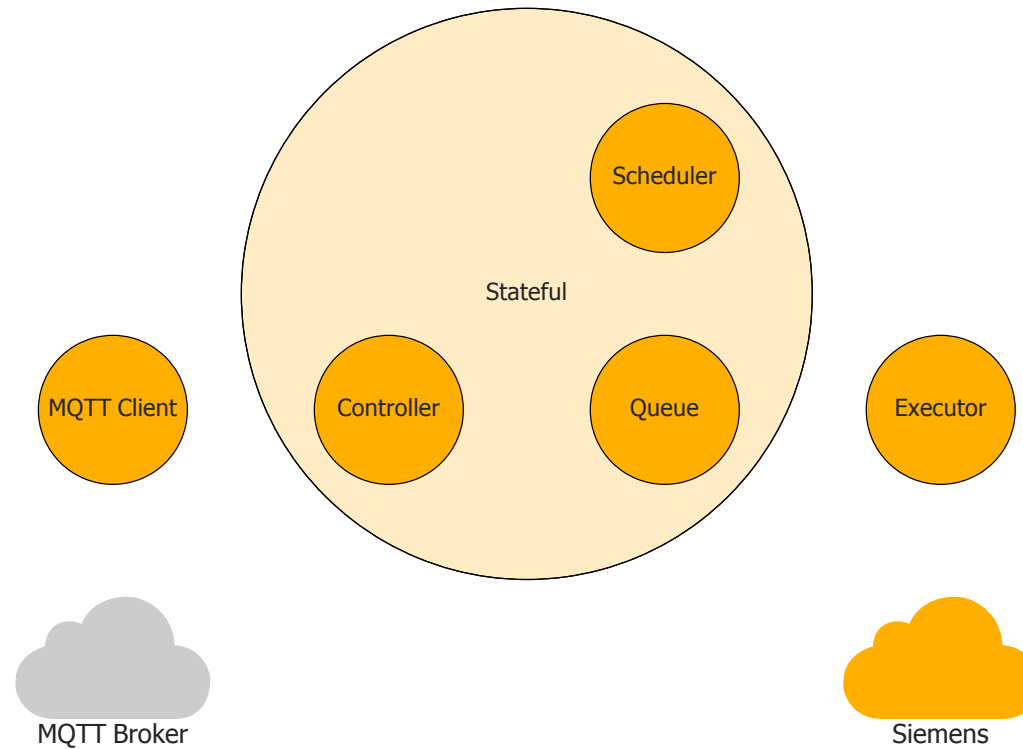
```
  {gen_server,call,
   [<0.13119.171>,
    {checkout,#Ref<0.2847797234.795344897.187204>,
     true,15000},
    5000]}}
```

in function 'Elixir.DBConnection.Poolboy':checkout/3 (lib/db_connection/poolboy.ex, line 112)

Takeaway - No-bangs are not inherently safe

- If a process holds important state, it not should talk to external systems
- A no-bang function is not safe from unintended errors
- Use tasks to handle these kinds of jobs
 - Processes are really cheap
 - If you're in doubt, spawn a new process

Mingling stateful processes



Takeaway - Use error kernels

- Keep nesting your supervision trees, to separate processes with important state from volatile processes
- See Error kernerls by Jesper Louis Andersen
- You probably can't have too much supervision

Returning errors

Two-tuple error responses

```
defmodule FIP.Siemens.Client do
  def get(url, params) do
    with {:ok, %HTTPoison.Response{status_code: 200} = response} <- do_get(url, params),
         {:ok, result} <- decode(response.body),
         {:ok, result} <- check_result(result) do
      {:ok, result}
    else
      {:ok, %HTTPoison.Response{} = response} -> {:error, {:http_status_code, response}}
      {:error, %HTTPoison.Error{} = error} -> {:error, {:http, error}}
      {:error, {:invalid, _, _} = error, body} -> {:error, {:json_decode, error, body}}
      {:error, {:invalid, _, _} = error, body} -> {:error, {:json_decode, error, body}}
      {:error, %{"Result" => _} = result} -> {:error, {:climatixic, result}}
    end
  end
end
```

Convert errors into Exception structs

```
defmodule FIP.Error do
  defexception reason: nil

  def exception(reason) do
    %__MODULE__{reason: reason}
  end

  def message(%__MODULE__{reason: reason}) do
    format_error(reason)
  end

  defp format_error({:http_status_code, response}),
    do: "HTTP|status_code|#{response.status_code}|#{response.request_url}"
end
```

Working with error structs

```
defmodule FIP.Siemens.Client do
  def get(url, params) do
    with {:ok, %HTTPoison.Response{status_code: 200} = response} <- do_get(url, params),
         {:ok, result} <- decode(response.body),
         {:ok, result} <- check_result(result) do
      {:ok, result}
    else
      error ->
        formatted_error = format_error(error)
        Logger.warn(Error.message(error))
        Metric.Error.observe(formatted_error)
        {:error, formatted_error}
    end
  end
end

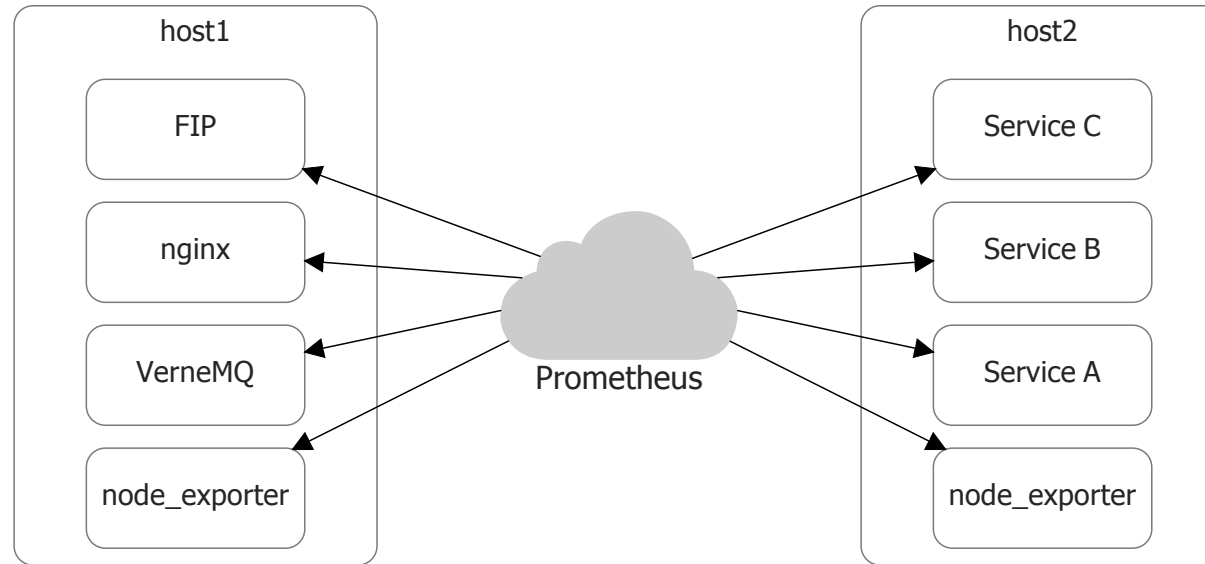
defp format_error({:ok, %HTTPoison.Response{} = response}), do: Error.exception({:http_status_code, response})
...
end
```

Takeaway - Use Exception structs

- The earlier you get started, the easier it will be
- Don't need to have a single, central Error module - for bigger apps, it could make sense to have several Error modules
- Check out Michał's blog post [Error handling in Elixir libraries](#)

Instrumentation and metrics

Metrics with Prometheus



What Prometheus collects

```
# TYPE http_response_duration_milliseconds histogram
# HELP http_response_duration_milliseconds HTTP response time
http_response_duration_milliseconds_bucket{host="ecodash.ecogrid.io",le="100"} 533172
http_response_duration_milliseconds_bucket{host="ecodash.ecogrid.io",le="500"} 853685
http_response_duration_milliseconds_bucket{host="ecodash.ecogrid.io",le="1000"} 877663
http_response_duration_milliseconds_bucket{host="ecodash.ecogrid.io",le="2000"} 882239
http_response_duration_milliseconds_bucket{host="ecodash.ecogrid.io",le="5000"} 884591
http_response_duration_milliseconds_bucket{host="ecodash.ecogrid.io",le="10000"} 886914
http_response_duration_milliseconds_bucket{host="ecodash.ecogrid.io",le="20000"} 888520
http_response_duration_milliseconds_bucket{host="ecodash.ecogrid.io",le="+Inf"} 890583
# TYPE erlang_vm_memory_atom_bytes_total gauge
# HELP erlang_vm_memory_atom_bytes_total The total amount of memory currently allocated for atoms...
erlang_vm_memory_atom_bytes_total{usage="used"} 883784
erlang_vm_memory_atom_bytes_total{usage="free"} 16257
```

Dashboards with Grafana



Custom metrics

```
defmodule FIP.Metric.Error do
  use Prometheus.Metric

  def setup() do
    Counter.new([
      name: :error_http_status_code,
      labels: [:host, :status_code],
      help: "HTTP responses with non-200 status code"
    ])
  end

  def observe(%FIP.Error{reason: {:http_status_code, response}}) do
    uri = URI.parse(response.request_url)
    Counter.inc([
      name: :error_http_status_code,
      labels: [uri.host, response.status_code]
    ])
  end
end
```

Hackney also lets you define metrics

```
defmodule FIP.Metric.Hackney do
  use Prometheus.Metric

  def setup() do
    Histogram.new([
      name: :http_response_duration_milliseconds,
      labels: [:host],
      buckets: [100, 500, 1_000, 2_000, 5_000, 10_000, 20_000],
      duration_unit: false,
      help: "HTTP response time"
    ])
  end

  def update_histogram([:hackney, host, :response_time], value) do
    value = trunc(value)
    Histogram.observe([
      name: :http_response_duration_milliseconds,
      labels: [host]
    ], value)
    :ok
  end
end
```

Takeaway - Do monitoring

- You get a lot for free with Prometheus
- Get a sense of what normal operation looks like, then set alerts

Deployment

Environment

- Single VM
- Debian Jessie
- Managed with Ansible
 - OS configuration
 - App configuration
 - Deployments

Tooling

- Releases
 - Distillery
 - Conform (to load config from init-style config file)
- Ansible
 - Home-grown Elixir build & deploy role
- Docker (build environment)
- Vagrant (testing deployment)
- System services
 - systemd
 - rsyslog
 - logrotate

Working with releases

- Setup
 - Use Distillery to generate release config
 - Add Conform plugin
 - Generate configuration schema and sample config file with Conform

Working with releases

- Setup
 - Use Distillery to generate release config
 - Add Conform plugin
 - Generate configuration schema and sample config file with Conform
- Deployment
 - Build the release with Distillery inside Docker
 - Ship release tar.gz file to server
 - Unarchive to proper directory
 - Make any necessary changes to config files

Working with releases

- Setup
 - Use Distillery to generate release config
 - Add Conform plugin
 - Generate configuration schema and sample config file with Conform
- Deployment
 - Build the release with Distillery inside Docker
 - Ship release tar.gz file to server
 - Unarchive to proper directory
 - Make any necessary changes to config files

```
ansible-playbook -i production -K --ask-vault-pass -e elixirapp_version=master roll.yml
```

Configuring a release

- The Mix config is baked into `sys.config`

```
[{sasl,[{errlog_type,error}]}],
{fip,
  [{'Elixir.FIP.Ecodash',
    [{pool_name,ecodash_pool},
     {pool_size,10},
     {token,<<"ohsnap">>},
     {url,<<"secret-url">>}]}],
  {'Elixir.FIP.MQTTClient',
    [{broker_host,<<"10.10.153.26">>},
     {broker_port,1883},
     {password,<<"ohsnap">>},
     {username,<<"uptime">>}]}],
...}
```

With Conform

```
# URL for Ecodash status update.
fip.Elixir.FIP.Ecodash.url = "secret-url"

# Token for Ecodash status update.
fip.Elixir.FIP.Ecodash.token = "ohsnap"

# Hostname of FIP MQTT broker.
fip.Elixir.FIP.MQTTClient.broker_host = "10.10.153.26"

# Port of FIP MQTT broker.
fip.Elixir.FIP.MQTTClient.broker_port = "1883"

# Username for FIP MQTT client.
fip.Elixir.FIP.MQTTClient.username = "uptime"

# Password for FIP MQTT client.
fip.Elixir.FIP.MQTTClient.password = "ohsnap"
```

All thanks to the schema

```
...  
  "fip.Elixir.FIP.MQTTClient.username": [  
    commented: false,  
    datatype: :binary,  
    default: "uptime",  
    doc: "Username for FIP MQTT client.",  
    hidden: false,  
    to: "fip.Elixir.FIP.MQTTClient.username"  
  ],  
  "fip.Elixir.FIP.MQTTClient.password": [  
    commented: false,  
    datatype: :binary,  
    default: "ohsnap",  
    doc: "Password for FIP MQTT client.",  
    hidden: false,  
    to: "fip.Elixir.FIP.MQTTClient.password"  
  ],  
  ...
```


Takeaway - Use releases

- Awesome tool which date back all the way to the beginning
- Adding configuration parameters can be painful
 - This is being adressed, maybe already in Elixir 1.7
 - Check out Paul Schoenfelders [Elixir Deployment Tools Update](#)
- Find a way to automate it

Debugging

:runtime_tools and dbg

```
iex(1)> :dbg.  
c/3  
ctp/0  
ctp/3  
ctpg/1  
ctpl/0  
ctpl/3  
dtp/0  
flush_trace_port/0  
get_info/0  
h/0  
ln/0  
match_front/2  
p/1  
start/0  
stop_trace_client/1  
tp/4  
tpl/3  
trace_client/3  
trace_port_control/2  
tracer/3  
wrap_presort/2  
  
c/4  
ctp/1  
ctpe/1  
ctpg/2  
ctpl/1  
deliver_and_flush/1  
dtp/1  
flush_trace_port/1  
get_tracer/0  
h/1  
ltp/0  
match_rear/2  
p/2  
stop/0  
tp/2  
tpe/2  
tpl/4  
trace_port/2  
tracer/0  
transform_flags/1  
wrap_sort/2  
  
cn/1  
ctp/2  
ctpg/0  
ctpg/3  
ctpl/2  
dhandler/2  
erlang_trace/3  
fun2ms/1  
get_tracer/1  
i/0  
match_0_9/1  
n/1  
rtp/1  
stop_clear/0  
tp/3  
tpl/2  
trace_client/2  
trace_port_control/1  
tracer/2  
wrap_postsrt/1  
wrap_sortfix/2
```

Takeaway - the tools are there

- You'll need some way to find your processes
- Remote shell and runtime_tools can get you a long way
- If you don't want to shoot yourself in the foot, look for a wrapper around the native tools
 - recon
 - Tracer

Retrospective

Things that worked well for us

- Elixir
- Keeping the application simple
 - Ship it fast
 - Now that we've felt some pain, we know how to structure the next version
- Using Exception structs for errors
- Using Prometheus for monitoring
- Using the Registry

Things we would have done differently

- Read the documentation more carefully
- More granular supervision
- Do more extensive testing of failure scenarios
- Write more docs & types
 - Mostly because we're not working in a familiar language/domain
- Separation of concerns inside GenServer
 - Separate public API, OTP callbacks and business logic
 - Takes a little more thinking upfront, but it's really minimal

Things we will be doing

- Management interface
- Use Phoenix.PubSub for service registration and inter-process communication
- Hot code reloading?
- Distribution?

Thank you 😊

I'm Troels Brødsgaard (troels@uptime.dk / @trarbr)

🙏 to Uptime for sending me and my colleagues